Markdown

Copy

```
# AI Modular Pipeline Framework
**A fully observable, node-based, provider-agnostic AI execution system**

---

## Mission Overview
This document defines the complete architecture, data formats, execution work

Every stage is **observable, inspectable, and editable live in the browser**.

---

# 1. System Concept

A modular, typed, observable AI pipeline with explicit stages:

1. Ingest messy human input (speech, text, or URLs)
2. Clean and normalize input
3. Convert into a structured, model-agnostic job specification
4. (Optional) Fetch external content (web pages, APIs)
5. Extract and summarize links/content when requested
6. Adapt the spec to a specific model provider
7. Execute and normalize provider output
8. Render every stage live in HTML for inspection and debugging

### Core Principles
- **Decomposition** – Every stage is a pure JSON → JSON function
- **Human-in-the-loop** – Users approve or edit key stages
- **Provider-agnostic** – Works with mock, Hugging Face, OpenAI, local models
- **Visual Debugging** – Full input/output JSON per node, live in browser
- **Web-Aware** – Built-in nodes for fetching public websites, crawling, link
- **Scalability** – Easily extended to multi-step reasoning, RAG, tool use, a

---

# 2. System Layers

| Layer                  | Responsibility
|------------------------|-----------------------------------------------
| **Input Layer**        | Raw speech, typed text, pasted URLs, or uploaded f
```

```
| **Pipeline Nodes**     | Chain of pure transformation nodes (typed JSON in
| **Tool / Web Layer**   | HTTP fetching, HTML parsing, link extraction, summ
| **Provider Layer**     | Adapters for Hugging Face, OpenAI, Groq, vLLM, Oll
| **Visualization Layer**| Live HTML panels showing each node's input, output

---

# 3. End-to-End Pipeline Examples

## Example 1 – Basic Text → Response (Default)

1. **STT / Text Input** → `RawSpeechText`
2. **Prompt Cleaner** → `PromptDraft`
3. **Prompt Approval** → `ApprovedPrompt`
4. **Spec Builder** → `ModelSpec`
5. **Model Adapter** → `ProviderRequest`
6. **Provider Execution** → raw response
7. **Response Formatter** → `ModelOutput`

## Example 2 – "Analyze this website" (with fetching & crawling)

**User says:** "Go look at https://news.ycombinator.com and tell me the top 5

### 1. STT / Text Input Node
```json
{
  "type": "RawSpeechText",
  "text": "go look at hacker news and tell me the top five stories right now"
  "meta": { "source": "mic", "language": "en-US", "timestamp": 1732912345 }
}
```

2. Prompt Cleaner Node
```

JSON


```
{
  "type": "PromptDraft",
  "original_text": "go look at hacker news and tell me the top five stories r
  "clean_text": "Fetch https://news.ycombinator.com and list the current top
  "detected_urls": ["https://news.ycombinator.com"],
  "meta": { "auto_cleaned": true }
}
```


3. Web Fetch Node (new tool node)

JSON


```
{
  "type": "FetchedPage",
  "url": "https://news.ycombinator.com",
  "status": 200,
  "content_type": "text/html",
  "text_content": "<!DOCTYPE html>...[full HTML]...",
  "meta": { "fetched_at": 1732912350, "fetch_duration_ms": 412 }
}
```


4. HTML → Links & Titles Extractor Node

JSON

Copy

```
{
  "type": "ExtractedLinks",
  "page_title": "Hacker News",
  "links": [
    {
      "title": "Show HN: I built a tiny database that runs in the browser",
      "url": "https://news.ycombinator.com/item?id=42123456",
      "points": 512,
      "comments": 189
    },
    {
      "title": "Linux 6.12 released",
      "url": "https://news.ycombinator.com/item?id=42123301",
      "points": 478,
      "comments": 312
    },
    {
      "title": "The era of cheap helium is over",
      "url": "https://news.ycombinator.com/item?id=42123099",
      "points": 398,
      "comments": 156
    },
    {
      "title": "OpenAI announces o3-mini",
      "url": "https://news.ycombinator.com/item?id=42122987",
      "points": 367,
      "comments": 201
    },
    {
      "title": "A new LLVM-based Rust compiler",
      "url": "https://news.ycombinator.com/item?id=42122845",
      "points": 342,
      "comments": 98
    }
  ],
  "meta": { "extracted_at": 1732912352 }
}
```

5. Spec Builder Node (now includes retrieved context)

-

JSON

Copy

```
{
  "type": "ModelSpec",
  "task_type": "web_analysis",
  "ai_role": "A helpful web research assistant",
  "primary_goal": "Summarize the top 5 stories on Hacker News right now",
  "retrieved_context": "{...ExtractedLinks JSON above...}",
  "input_prompt": "Using the provided Hacker News front page data, list the c
  "model_behavior": { "style": "concise", "temperature": 0.2, "max_tokens": 5
  "provider": { "name": "openai", "model": "gpt-4o-mini" }
}
```

6–8. Model Adapter → Execution → Formatter (standard)

Final output displayed to user with sources and live links.

---

4. File / Module Architecture (Updated)

```text
project-root/
│
├── index.html
├── styles.css
│
├── js/
│   ├── main.js                    # Orchestrator + UI updates
│   ├── pipeline-config.js         # Named pipelines + node order
│   ├── ui/
│   │   ├── panel-renderer.js
│   │   └── approval-widget.js
│   │
│   ├── nodes/
│   │   ├── node-stt.js
│   │   ├── node-prompt-cleaner.js
│   │   ├── node-prompt-approval.js
│   │   ├── node-web-fetch.js          # NEW: fetches public URLs
│   │   ├── node-html-parser.js        # NEW: extracts links, titles, metada
│   │   ├── node-spec-builder.js
│   │   ├── node-model-adapter.js
│   │   ├── node-response-formatter.js
│   │   └── node-summary.js            # Optional post-processing
│   │
│   └── providers/
│       ├── huggingface.js
│       ├── openai.js
│       ├── groq.js
│       ├── ollama.js
│       └── mock.js
│
└── assets/
```

---

5. Key New Nodes (Web-Aware)

node-web-fetch.js

- Accepts any public HTTP/HTTPS URL
- Performs `fetch()` with `cors` handling fallback via public proxy if needed
- Returns full text/html or JSON when detectable
- Respects robots.txt (optional toggle)

`node-html-parser.js`

- Extracts:
  - `<title>`
  - All `<a href>` with readable text
  - OpenGraph / Twitter card metadata
  - HN-specific ranking if detected
- Outputs structured `ExtractedLinks` with descriptions

`node-spec-builder.js` (enhanced)

- Detects presence of URLs → automatically inserts fetch/extract nodes
- Injects retrieved context into the final prompt
- Supports multiple pages and recursive crawling (depth-limited)

---

6. Plug-and-Play & Extensibility

Feature	How to Add / Swap
New LLM provider	Add file in `providers/` , update adapter mapping
New tool (search, calculator, etc.)	Add node → register in pipeline config
Crawling depth >1	Chain multiple `web-fetch + html-parser` nodes
RAG / vector search	Add `node-vector-retrieve` and merge into context
Image generation/analysis	Add vision-specific spec + provider adapters

---

7. Roadmap (Updated — Nov 2025)

Already done / In Progress:

1. Complete base HTML + live JSON panels
2. Real browser STT (Web Speech API)
3. Robust prompt cleaning + URL detection
4. Web fetching + HTML link extraction nodes
5. OpenAI + Hugging Face + Ollama providers

Next:

- ☐ Recursive crawler with depth limit & visited-set
- ☐ Built-in simple vector store for session memory
- ☐ Tool-use / function-calling pipeline (parallel tools)
- ☐ Export/import pipeline runs as JSON
- ☐ Graph (DAG) mode instead of linear only
- ☐ Mobile PWA support

---

8. Conceptual Summary

This framework is a **general-purpose AI operations engine** that:

- Turns vague, messy, or web-aware human requests into fully structured, observable AI jobs
- Seamlessly fetches public websites, crawls them, extracts and describes links
- Keeps humans in control at every critical step
- Remains completely provider- and model-agnostic
- Offers live, visual debugging of every transformation

Transparent • Deterministic • Extensible • Web-Integrated • Human-Centric

A complete, production-ready foundation for building the next generation of observable, tool-using AI applications directly in the browser.

↳ Add recursive crawling example

↳ Integrate vector database node

↳ more implementation code snippets